

Home ▶ STM32 Tutorials ▶ FreeRTOS Tutorials ▶ **Mutex: Priority Inheritance and Recursive**

Updated On: May 6, 2026

STM32 FreeRTOS Mutex: Priority Inheritance and Recursive Mutex

In the [previous tutorial](#), we covered semaphores in FreeRTOS and saw how they can be used to protect shared resources between tasks. We also saw a problem called **priority inversion**, where a high priority task ends up waiting for a low priority task, and a medium priority task makes things even worse by preempting the low priority task in between.

In this tutorial, we will solve that problem using a **mutex**. A mutex works similarly to a semaphore, but it has one key advantage — it implements **priority inheritance**, which prevents priority inversion from happening.

We will also cover **recursive mutexes**, which solve another problem called deadlock that can occur when the same task tries to acquire a mutex more than once.

Here is what we will cover in this tutorial:

- Why a mutex is better than a semaphore for protecting shared resources
- How priority inheritance works and how it prevents priority inversion
- What a recursive mutex is and when to use it

This is the **Part 5** of the **STM32 CMSIS-RTOS FreeRTOS series**. You can

[HOME](#)[STM32](#) ▼[ESP32](#) ▼[Arduino](#) ▼

- **Part 1** — Configure CMSIS RTOS in STM32
- **Part 2** — Creating Multiple Tasks and Priorities in CMSIS RTOS
- **Part 3** — How to use Queues for inter-task communication
- **Part 4** — Using Semaphores in CMSIS RTOS
- **Part 6** — Task Synchronization using Event Flags
- **Part 7** — Software Timers in FreeRTOS
- **Part 8** — FreeRTOS Stack Management

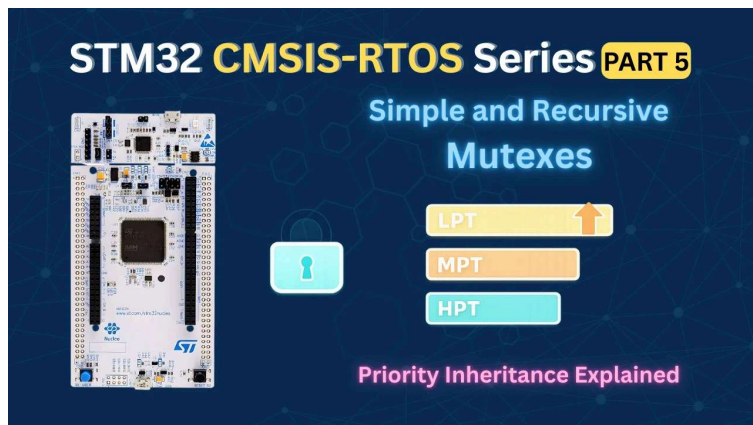


Table Of Contents

Mutex vs Semaphore: How Priority Inheritance Works

A mutex (short for **Mutual Exclusion**) is a synchronization primitive used to protect shared resources in a multitasking system. Just like a semaphore, only one task can hold the mutex at a time. Any other task that tries to acquire it will be blocked until the mutex is released.

But a mutex is not just a semaphore with a different name. It has a very important feature that a semaphore does not : **priority inheritance**. This is what makes a mutex the right tool for protecting shared resources between tasks of different priorities.

What Makes a Mutex Different from a Semaphore

On the surface, a mutex and a binary semaphore look very similar. Both allow only one task to access a shared resource at a time. But there is one fundamental difference between them.

A semaphore has no concept of ownership. Any task can release a semaphore, even if it was not the one that acquired it. A mutex, on the other hand, has ownership, therefore, only the task that acquired the mutex can release it.

This ownership concept is what enables priority inheritance, which we will see in a moment.

The table below shows the comparison between semaphore and mutex:

Feature	Semaphore	Mutex
Ownership	No	Yes
Priority Inheritance	No	Yes
Default State	Available or Depleted	Always Available
Recursive Acquire	No	Only with Recursive Mutex
Use Case	Task Signaling	Resource Protection

How Does Priority Inheritance Work?

Let's say we have three tasks — a low priority task (LPT), a medium

1. LPT acquires the mutex and starts using the shared resource.
2. HPT wakes up and tries to acquire the mutex, but it is already held by LPT, so HPT has to wait.
3. MPT wakes up and since it does not need the mutex, it preempts LPT and starts running.
4. Now LPT cannot release the mutex because MPT is occupying the CPU.
5. HPT is stuck waiting, not because of the mutex, but because MPT is indirectly blocking LPT.

This is priority inversion. Here HPT is waiting while MPT is running, which effectively reverses their priorities.

The image below demonstrates the priority inversion caused while using a semaphore.

With a mutex, FreeRTOS solves this automatically using **priority inheritance**. The moment HPT starts waiting for the mutex, FreeRTOS temporarily raises the priority of LPT to match HPT. This means LPT now has a higher priority than MPT, so MPT cannot preempt it anymore.

The diagram below illustrates this:

Once LPT releases the mutex, its priority is immediately dropped back to its original value. This priority boost is temporary and automatic and FreeRTOS handles it internally.

This is the core reason why you should always use a mutex instead of a semaphore when protecting shared resources between tasks of different priorities.

What Is a Recursive Mutex and When to Use It

A standard mutex works well for most cases, but it has one limitation — the same task cannot acquire it more than once. If a task tries to acquire a mutex it already holds, it will deadlock and freeze forever. This is where the recursive mutex comes in.

The Problem of Deadlock with a Simple Mutex

Imagine a task that calls `fs_write()` to write data to a file. To protect the shared filesystem, `fs_write()` acquires a mutex before doing anything.

But internally, `fs_write()` calls `fs_allocate_block()` to allocate space on the filesystem, which in turn calls `fs_update_metadata()` to update the filesystem records. Both of these functions also try to acquire the same mutex to protect the shared resource.

```
1 fs_write()           // acquires mutex
2   └─ fs_allocate_block() // tries to acquire the same mutex
3     └─ fs_update_metadata()
```

1. `fs_write()` acquires the mutex successfully.
2. It calls `fs_allocate_block()` , which tries to acquire the same mutex.
3. The mutex is already held by the same task, so `fs_allocate_block()` blocks and waits.
4. But the task is the one holding the mutex, so it can never release it.
5. The task waits forever and this is a deadlock.

The simple mutex has no concept of how many times the same task has acquired it. It just sees that the mutex is taken and blocks the caller, even if the caller is the one holding it.

How Recursive Mutex Solves the Deadlock

A recursive mutex solves this by keeping an internal acquisition count. Every time the owning task acquires it again, the count increases. Every time the task releases it, the count decreases. The mutex is only truly released and made available to other tasks when the count reaches zero.

```

1 fs_write()           // acquires recursive mutex → coun
2 └─ fs_allocate_block() // acquires recursive mutex → coun
3   └─ fs_update_metadata()
4     // fs_update_metadata returns → count = 2
5 // fs_allocate_block releases → count = 1
6 // fs_write releases → count = 0 → mutex now available to o

```



The rule is simple, for every `osMutexAcquire()` call, there must be a matching `osMutexRelease()` call. If a task acquires the recursive mutex three times, it must also release it three times before another task can acquire it.

This makes the recursive mutex the right choice whenever you have nested function calls that all need to protect the same shared resource.

CubeMX Setup: Tasks and Mutex Configuration

Before we write any task code, we need to configure the mutex in STM32CubeMX. We need four tasks and one mutex for this part.

Configuring the Four Tasks

Go to Middleware → FreeRTOS and open the Tasks and Queues tab. We

and MPT will simply run without needing the mutex. VHPT will print the priorities of all the other tasks periodically.

The image below shows the four tasks configured in CubeMX:

Here is a quick breakdown of each task:

- **LPT** is the low priority task. Set its priority to **Below Normal** and stack size to **512 words**. This task will acquire the mutex and

[HOME](#)[STM32](#) ▾[ESP32](#) ▾[Arduino](#) ▾

- **MPT** is the medium priority task. Set its priority to **Normal** and stack size to **512 words**. This task will try to preempt LPT while it is holding the mutex.
- **HPT** is the high priority task. Set its priority to **Above Normal** and stack size to **512 words**. This task also needs the mutex to run, so it will be blocked whenever LPT holds it.
- **VHPT** is the Very high priority task. Set its priority to **High** and stack size to **512 words**. This task will print the priorities of the other tasks periodically.

We are using 512 words for all three tasks because we will be printing logs using `printf`, which needs a decent amount of stack space.

Adding a Standard Mutex in CubeMX

Now go to the Mutex tab. Here you will see two types of mutexes available:

- **Mutex** — the standard mutex with priority inheritance
- **Recursive Mutex** — a mutex that the same task can acquire multiple times

Let's start with adding a standard mutex.

You can leave the name as `myMutex01` . Make sure the allocation is set to **Dynamic** so it takes memory from the FreeRTOS heap.

Note: After configuring tasks and mutexes, you might see a heap size error. Go to Config Parameters and increase the Total Heap Size to around 10000 bytes. The error should go away.

Once the code is generated, open the `main.c` file. You will see that CubeMX has added the following line to initialize the mutex:

```
1 myMutex01Handle = osMutexNew(&myMutex01_attributes);
```

The `osMutexNew()` function creates the mutex and returns a handle that

the acquire function to acquire it. Unlike a semaphore, where we had the option to make it either available or depleted by default, a mutex is always available by default.

Adding a Recursive Mutex in CubeMX

Under the Recursive Mutex section, click Add to add the mutex.

You can leave the name as `myRecursiveMutex01` . Make sure the allocation is set to **Dynamic** so it takes memory from the FreeRTOS heap.

Once the code is generated, you will see the recursive mutex initialized in `main.c` :

Configuring LPUART1 for printf Output

We will use `printf` to print the received data to a serial console. On the STM32L496 Nucleo board, the Virtual COM port routes UART data through the ST-LINK USB connection. Looking at the board schematic, ST-LINK RX is connected to **LPUART1 TX** and ST-LINK TX is connected to **LPUART1 RX**. These map to pins **PG7** and **PG8**.

Go to Connectivity and enable LPUART1 in Asynchronous mode. By default, CubeMX assigns pins PC0 and PC1, so change these to **PG7 (TX)** and **PG8 (RX)**.

Use the following settings:

Parameter	Value
Baud Rate	115200
Word Length	8 bits
Parity	None
Stop Bits	1

This is the standard UART configuration and matches what we will set in the serial viewer.

Code & Results: Mutex and Recursive Mutex

Task Code: LPT, MPT, HPT, and VHPT

We have four tasks in this project. Let's go through each one and write the code. The overall idea is simple, the low priority task holds the mutex while simulating resource usage, the high priority task needs the mutex to run, and the medium priority task runs freely without needing the mutex at all. The very high priority task just monitors the priorities of all three tasks so we can observe priority inheritance in action.

Low Priority Task – Acquiring and Releasing the Mutex

The low priority task is where the interesting part happens. It acquires the mutex, simulates resource usage by running an empty loop for around one second, and then releases the mutex.

```

1 void StartLPT(void *argument)
2 {
3     uint32_t wait;
4     for(;;)
5     {
6         printf("Entered LPT, waiting for Mutex\n\n");
7         osMutexAcquire(myMutex01Handle, osWaitForever);
8         printf("LPT Acquired Mutex, using Resource\n\n");
9         wait = 10000000;
10        while (wait--);
11        printf("LPT finished, released Mutex\n\n");
12        osMutexRelease(myMutex01Handle);
13        printf("LPT going to Sleep\n\n");
14    }
15 }
```

We use `osWaitForever` as the timeout so the task blocks indefinitely until the mutex becomes available. Once it acquires the mutex, it runs the empty `while` loop to simulate one second of resource usage. After that, it calls `osMutexRelease()` to release the mutex and let other tasks acquire

Medium Priority Task – Running Without the Mutex

The medium priority task does not need the mutex at all. It simply runs its empty loop for around five seconds and then goes to sleep.

```
1 void StartMPT(void *argument)
2 {
3     uint32_t wait;
4     for(;;)
5     {
6         printf("Entered MPT, performing task\n\n");
7         wait = 50000000;
8         while (wait--);
9         printf("MPT finished, going to Sleep\n\n");
10        osDelay(500);
11    }
12 }
```

This task is the one that causes priority inversion when using a semaphore. Since it does not need the mutex, it can freely preempt the low priority task. But as we will see, the mutex prevents this from being a problem.

High Priority Task – Waiting for the Mutex

The high priority task needs the mutex to run its process.

```
1 void StartHPT(void *argument)
2 {
3     for(;;)
4     {
5         printf("Entered HPT, waiting for Mutex\n\n");
6         osMutexAcquire(myMutex01Handle, osWaitForever);
7         printf("HPT Acquired and now releasing Mutex\n\n");
8         osMutexRelease(myMutex01Handle);
9         printf("HPT going to Sleep\n\n");
10        osDelay(200);
11    }
12 }
```

It acquires the mutex, prints its success message, and immediately releases it before going to sleep for 200 milliseconds.

Very High Priority Task – Monitoring Task Priorities

This task runs every 50 milliseconds and prints the current priorities of all three tasks. Since priority inheritance temporarily changes the priority of the low priority task, this task lets us observe that change in real time.

```

1 void StartVHPT(void *argument)
2 {
3     for(;;)
4     {
5         printf("Priorities →→ LPT:%d MPT:%d HPT:%d\n\n",
6               osThreadGetPriority(LPTHandle),
7               osThreadGetPriority(MPTHandle),
8               osThreadGetPriority(HPTHandle));
9         osDelay(50);
10    }
11 }

```

We use `osThreadGetPriority()` to fetch the current priority of each task. Normally, LPT has a priority of 16, MPT has 24, and HPT has 32. But when HPT starts waiting for the mutex, you will see LPT's priority temporarily jump to 32. This task runs every 50 milliseconds, so we will be able to observe the priority changes in real time.

Here is the full task code combined:

```

1 void StartLPT(void *argument)
2 {
3     uint32_t wait;
4     for(;;)
5     {
6         printf("Entered LPT, waiting for Mutex\n\n");
7         osMutexAcquire(myMutex01Handle, osWaitForever);
8         printf("LPT Acquired Mutex, using Resource\n\n");
9         wait = 100000000;

```

```
13     printf("LPT going to Sleep\n\n");
14 }
15 }
16
17 void StartMPT(void *argument)
18 {
19     uint32_t wait;
20     for(;;)
21     {
22         printf("Entered MPT, performing task\n\n");
23         wait = 50000000;
24         while (wait--);
25         printf("MPT finished, going to Sleep\n\n");
26         osDelay(500);
27     }
28 }
29
30 void StartHPT(void *argument)
31 {
32     for(;;)
33     {
34         printf("Entered HPT, waiting for Mutex\n\n");
35         osMutexAcquire(myMutex01Handle, osWaitForever);
36         printf("HPT Acquired and now releasing Mutex\n\n");
37         osMutexRelease(myMutex01Handle);
38         printf("HPT going to Sleep\n\n");
39         osDelay(200);
40     }
41 }
42
43 void StartVHPT(void *argument)
44 {
45     for(;;)
46     {
47         printf("Priorities →→ LPT:%d MPT:%d HPT:%d\n\n",
48             osThreadGetPriority(LPTHandle),
49             osThreadGetPriority(MPTHandle),
50             osThreadGetPriority(HPTHandle));
51         osDelay(50);
52     }
53 }
```



Now that we have written the task code, let's flash the project to the board and observe the output on the serial monitor. This is where we can actually see priority inheritance happening in real time.

The image below shows the full output on the serial monitor with annotations explaining each step.

Let's walk through what is happening step by step.

The output starts with the very high priority task printing the priorities of all three tasks — LPT is 16, MPT is 24, and HPT is 32. After that, MPT runs its empty loop and goes to sleep. Once MPT finishes, LPT gets the CPU, acquires the mutex, and starts using the resource.

At this point the priorities are still the same. Since LPT has a lower priority

But then HPT wakes up and tries to acquire the mutex. The mutex is already held by LPT, so HPT has to wait. The moment HPT starts waiting, something interesting happens — the priority of LPT instantly jumps from 16 to 32. **This is priority inheritance in action.** FreeRTOS automatically raised LPT's priority to match HPT's priority.

Now LPT has a priority of 32, which is higher than MPT's priority of 24. So MPT cannot preempt LPT anymore. LPT finishes its resource usage and releases the mutex. As soon as the mutex is released, HPT acquires it and runs its process.

You can also see that once LPT releases the mutex, its priority immediately drops back to 16. The priority boost was only temporary — it lasted just long enough for LPT to finish and release the mutex. After that, MPT enters the CPU and starts its task as usual.

This is exactly how priority inheritance is supposed to work. The mutex handled the entire thing automatically without any extra code on our part.

Recursive Mutex Task Code

I will only use of the Tasks (LPT in this case) to demonstrate the Recursive mutex. The rest of the tasks created earlier are commented out.

Here we will add two functions — `FunctionA` and `FunctionB`.

`FunctionA` runs first, acquires the recursive mutex, and then calls

`FunctionB`. `FunctionB` also tries to acquire the same recursive mutex.

```
1 void FunctionB(void)
2 {
3     printf("FunctionB trying to lock mutex\n");
4     osMutexAcquire(myRecursiveMutex01Handle, osWaitForever
5     printf("FunctionB acquired mutex\n");
6     osDelay(100);
7     osMutexRelease(myRecursiveMutex01Handle);
8     printf("FunctionB released mutex\n");
```

```

12 {
13     printf("FunctionA trying to lock mutex\n");
14     osMutexAcquire(myRecursiveMutex01Handle, osWaitForever
15     printf("FunctionA acquired mutex\n");
16     FunctionB(); // tries to lock the same mutex again
17     osMutexRelease(myRecursiveMutex01Handle);
18     printf("FunctionA released mutex\n\n");
19 }

```

Here is what happens step by step:

1. FunctionA acquires the recursive mutex — count goes to 1.
2. FunctionA calls FunctionB , which acquires the same mutex again — count goes to 2.
3. FunctionB finishes its work and releases the mutex — count drops back to 1.
4. Control returns to FunctionA , which releases the mutex — count drops to 0.
5. The mutex is now fully available to other tasks.

With a simple mutex, step 2 would have caused a deadlock. But with the recursive mutex, the same task can acquire it multiple times without any issue, as long as it releases it the same number of times.

Inside the low priority task, we simply call FunctionA every 5 seconds.

```

1 void StartLPT(void *argument)
2 {
3     for(;;)
4     {
5         FunctionA();
6         osDelay(5000);
7     }
8 }

```

The image below shows the output on the serial monitor.

FunctionA acquires the mutex first, then calls FunctionB which also acquires the same mutex successfully. FunctionB releases it first, then FunctionA releases it, bringing the count back to 0. No deadlock here, everything works exactly as expected.

STM32 FreeRTOS Mutex — Priority Inheritance and Recursive Mutex Video Tutorial

This video walks through the full

[HOME](#)[STM32](#) ▾[ESP32](#) ▾[Arduino](#) ▾[TIVA.C](#) ▾
English ▾

configure the standard mutex and recursive mutex in STM32CubeMX, write task code for LPT, MPT, HPT, and VHPT, observe priority inheritance in real time on the serial console, and demonstrate how a recursive mutex prevents deadlock in nested function calls.

STM32 FreeRTOS Mutex — Frequently Asked Questions

- + Can I use a mutex from an interrupt service routine (ISR)?
- + What happens if a task forgets to release the mutex?
- + Does priority inheritance work automatically or do I need to enable it?
- + Can multiple tasks acquire the same mutex at the same time?
- + When should I use a recursive mutex over a standard mutex?

Conclusion

This tutorial covered both mutex types available in FreeRTOS — the standard mutex with priority inheritance, and the recursive mutex for nested locking scenarios.

With the standard mutex, we saw priority inheritance resolve priority inversion automatically. The moment HPT started waiting for the mutex held by LPT, FreeRTOS raised LPT's priority to match HPT's — preventing MPT from occupying the CPU and starving the chain. As soon as LPT released the mutex, its priority dropped back to normal. No extra code was needed; switching from a semaphore to a mutex was enough.

With the recursive mutex, we demonstrated how nested function calls can acquire the same lock multiple times without deadlocking. The internal acquisition counter ensures the mutex is only truly released when every matching `osMutexRelease()` has been called. This makes it the right choice for library code or layered drivers where the same resource needs protection across multiple call depths.

The rule of thumb is straightforward: use a standard mutex whenever tasks of different priorities share a resource, and use a recursive mutex when a single task may re-enter a locked section through nested calls.

The next tutorial in this series covers [FreeRTOS Event Flags](#) — a mechanism for synchronizing multiple tasks on a set of conditions simultaneously, which is more flexible than a binary signal but lighter than a queue. Download the full STM32CubeIDE project using the link below, and leave any questions in the comments.

Files



Complete STM32CubeIDE project with standard mutex and recursive mutex demos, all four task functions, priority inheritance output, and LPUART1 printf logging. Free to download — **support the work** if it helped you.

[Open source](#)[Mutex + Recursive Mutex](#)[Priority Inheritance Demo](#)[CubeMX + HAL source](#)[View on GitHub](#)[Support my work](#)

Browse More STM32 FreeRTOS Tutorials

[HOME](#)[STM32](#) ▾[ESP32](#) ▾[Arduino](#) ▾[TIVA.C](#) ▾
English ▾

**STM32 FreeRTOS
Tutorial: CMSIS-
RTOS V2 Setup,
Tasks & LED Blink
in CubeMX**

**STM32 FreeRTOS
Multiple Tasks,
Priorities &
Preemption —
CMSIS-RTOS V2
Guide**

**STM32 FreeRTOS
Queue Tutorial:
Inter-Task
Communication
with CMSIS-OS V2**

**STM32 FreeRTOS
Semaphores: How
to Use Binary and
Counting
Semaphores**

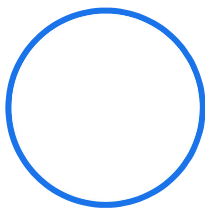
**STM32 FreeRTOS
Event Flags:
osFlagsWaitAll,
WaitAny & ISR
Callbacks**

**STM32 FreeRTOS
Software Timers:
Periodic & One-
Shot with CMSIS-
OS V2**

1 2 Next »



ABOUT THE AUTHOR



Arun Rawat

EMBEDDED SYSTEMS
ENGINEER · FOUNDER,
CONTROLLERSTECH

Arun is an embedded systems
engineer with 10+ years of

created ControllersTech to share practical tutorials on embedded software, HAL drivers, RTOS, and hardware design — grounded in real industrial automation experience.

[About](#)[YouTube](#)[GitHub](#)[Facebook](#)[Instagram](#)[Shop](#)[✉ Subscribe ▼](#)[Login](#)

{
[+]



0 COMMENTS

Subscribe to Our Newsletter

Email

☐ I accept the privacy policy

Subscribe



© 2026 ControllersTech® · All Rights Reserved · Built with ❤️ for Embedded Engineers

[About Me](#)

[Privacy Policy](#)

[SHOP](#)

[Contact US](#)

[Do Not Sell or Share My Personal Information](#)